Contents

1 Introduction **4**

2 Challenges of Typing JavaScript **7**

2.1 Typing for JavaScript 7

2.2 JavaScript behavior 7

2.2.1 Properties can be added/removed at runtime 8

2.2.2 Properties of a class may depend on the constructor . . 10

2.2.3 TypeScript can't use generics with primitive operations 11

2.2.4 Existence of multiple correct typings 12

2.3 Migration after types have been assigned 14

3 Related work **16**

3.1 Static analysis 16

3.2 Machine learning based analysis 16

3.3 Dynamic analysis 18

4 Implementation **19**

4.1 Preprocessing 19

4.1.1 Transformations 20

4.2 Watched Objects 21

4.2.1 Functions 22

4.2.2 Objects 23

4.3 Data Collection 24

4.3.1 Collecting Traces 24
4.3.2 Trace Sets 25
4.3.3 TypeInfo Objects 28
4.4 Post-processing 28
4.4.1 Computed Types 29
4.4.2 Computing Generic Types 31
4.4.3 Combining Types 32
4.4.4 TypeScript Generation 33
4.5 Usage 33
4.6 Evaluation 34

5 Results **35**

5.1 Research Question 1: Overhead 35
5.1.1 Speed 36
5.1.2 Memory consumption 37
5.1.3 Research Question 1: Summary 38
5.2 Research Question 2: Litmus Tests 39
5.2.1 Object assignment 40
5.2.2 Complex nested generics 42
5.2.3 Simple addition 45
5.2.4 Generators 47
5.2.5 Research Question 2: Summary 48
5.3 Research Question 3: Full package evaluation 49

5.3.1 abab 49
5.3.2 dotenv 51
5.3.3 node-int64 51
5.3.4 ini 51
5.3.5 Research Question 3: Summary 53

6 Conclusion **54**

6.1 Future Work 54
6.2 Artifacts 55

7 References **56**

1 Introduction

Typed programs are both more readable and less error-prone: function parameters types don't have to be guessed from their use and their name, and static analysis can determine type errors before program execution. It can be difficult, however, given a large untyped codebase, for developers to add types so late in development or to migrate from an untyped version of a language to a typed one. As a result, developers frequently rely on automated type inference tools [21].

For JavaScript this issue is particularly acute—TypeScript was released 17 years after JavaScript and has yet to fully supplant JavaScript even now, so there are many massive JavaScript programs that would be unwieldy to manually type, motivating the development of automated type inference engines.

There are three primary methods of type inference: static type analysis, AI-based inference and dynamic type inference. Of these, only the first two have been substantially explored in literature for JavaScript.

Static analysis typically uses data flow analysis to infer as many types as it can from the code, either transforming a program's AST into some lower-level representation more amenable to data flow analysis [11, 12] or working with a simpler subset of the language more conducive to static reasoning [2].

There are a variety of methods for AI-based type inference, from infilling

identifier types with LLMs [6], inferring types from surrounding documentation [13, 17], using more bespoke architectures [10, 20, 24], or simply putting the entire codebase into an LLM and having it directly apply types, though this may not preserve program behavior.

Unlike static or AI-based approaches, dynamic type inference inspects the types of a program during execution. A dynamic type inference tool instruments programs to intercept and log assignments to identifiers. This results in each identifier having a pool of distinct types the program has assigned to it, which will be an improper subset of the true types that identifier can take.

Each method has drawbacks. Static analysis tends to be slower on larger programs with more complex data flow paths [22]. AI-based type inference is inherently probabilistic and much of the existing work does not support complex types [6, 10, 24]. LLM-based type inference via infilling is not guaranteed to produce valid types, and a wholesale LLM migration may end up taking liberties when modifying the source code. While static analysis finds a superset of correct types and dynamic analysis finds a subset, we have no such guarantees about probabilistic approaches (Figure 1).

Before:

```
function EventEmitter() {
  EventEmitter.init.call(this);
}
// Backwards-compat with node 0.10.x
EventEmitter.EventEmitter = EventEmitter;
EventEmitter.prototype._events = undefined;
EventEmitter.prototype._eventsCount = 0;
EventEmitter.prototype._maxListeners = undefined;

Object.defineProperty(EventEmitter, 'defaultMaxListeners', {
  enumerable: true,
  get: function() { ... },
  set: function(arg) { ... }
});
```

After:

```
class EventEmitter {
  _events: EventsMap | undefined = undefined;
  _eventsCount = 0;
  _maxListeners: number | undefined = undefined;

  /** Backwards-compat with node 0.10.x (`require('events').EventEmitter`). */
  static EventEmitter: typeof EventEmitter;

  static get defaultMaxListeners(): number { ... }
  static set defaultMaxListeners(arg: number) { ... }

  constructor() {
    EventEmitter.init.call(this);
  }
}

// Backwards-compat with node 0.10.x
EventEmitter.EventEmitter = EventEmitter;
```

Figure 1: Claude Opus 4.7 migrating the events node package using a prompt similar to that in Section 5.2.5 (artifact). Claude migrates old-style object property definitions when only asked to convert JavaScript to TypeScript, and fails to adequately retain Node 0.10.x backwards compatibility, as native classes were introduced in Node 4.0.0 (Sections 1, 5.2.5).

2 Challenges of Typing JavaScript

2.1 Typing for JavaScript

There have been multiple approaches to applying types to variables in JavaScript. Of them this work will focus on TypeScript, as it is by far the most popular. TypeScript is syntactically a superset of JavaScript [4] and is transpiled to JavaScript before execution. One alternative is JSDoc [14], a standardized method of writing function documentation, which allows for specifying both parameter and return types of functions and is the standard for JavaScript programs. While JSDoc is primarily used for generating documentation websites , its type annotations can be checked by the TypeScript compiler, but TypeScript is generally preferable over JSDoc for conciseness and readability (Figure 2). Another alternative is Flow [5], but it is much less popular than TypeScript.

2.2 JavaScript behavior

JavaScript is a particularly difficult language to apply strict types to, as it was not designed with strict types in mind. TypeScript was created to minimize runtime type errors and have better code clarity [9]. While TypeScript is a strict superset of JavaScript, there exists JavaScript code that TypeScript has difficulty annotating [4, 7].

Existing JavaScript type inference engines are not full migration tools, typ-

TypeScript

```
function add(a: number, b: number): number { return a + b; }
```

JSDoc

```
/**
 * @param {number} a
 * @param {number} b
 * @returns {number}
 */
function add(a, b) { return a + b; }
```

Figure 2: A simple function whose types are annotated both by TypeScript and JSDoc. TypeScript only requires a single line whereas JSDoc requires 6 (Section 2.1).

ically simply predicting types for variables rather than performing a full migration. The issues described in this section only apply when translating from JavaScript to TypeScript, and thus are left unaccounted for by existing type inference engines.

2.2.1 Properties can be added/removed at runtime

While the TypeScript paradigm prefers an object to have a fixed set of properties at the moment it is defined, in JavaScript, the properties of a given class depend on the constructor, and an object can gain or lose properties at any time. It is common practice for objects to be initialized without properties at all. JavaScript is a dynamically typed language and allows for the types of identifiers to change on the fly, whereas TypeScript is statically typed and does not allow this to happen.

For the TypeScript compiler not to throw an error, the programmer must tell it the type of all properties it will have in that scope, even if they have yet to be defined. Here we have an example TypeScript program, typed idiomatically:

```
let myobj: { a: number, b: number, c?: number } = { a: 1, b: 1 }
function addc(obj: { a: number, b: number, c?: number }):
    { a: number, b: number, c?: number } {
  obj.c = 1
  return obj
}
addc(myobj)
console.log(myobj.c as number + 5)
```

These types are unhelpful. We would like to know for certain that `obj.c` has been set after `addc` has been called, but in idiomatic TypeScript, an object's type must stay constant. This results in the type of `myobj` to be {`a: number, b: number, c?: number`} throughout its lifetime. Thus, `as number` must be added to `myobj.c` as the TypeScript compiler does not update the type of `myobj` from {`a: number, b: number`}.

Here we have the same program, but we have asserted the existence of the property `c` after the function is called, allowing the programmer to skip the type assertion:

```
let myobj: { a: number, b: number } = { a: 1, b: 1 }
function addc(obj: { a: number, b: number, c?: number }):
    { a: number, b: number, c: number } {
  obj.c = 1
  return obj as { a: number, b: number, c: number }
```

```
}
let myobj2 = addc(myobj)
console.log(myobj2.c + 5)
```

Even in this case we still need to make a type assertion. Had this program been written in idiomatic TypeScript from the start there would be no need for type assertions, but since this is a conversion from JavaScript, there are some cases where it is impossible to output TypeScript code without the need for any type coercions.

In the idiomatic code, `myobj` is redefined as `myobj2`. This is because, while the actual object `myobj` certainly has the `c` property and the result of `addc` certainly has the `c` property, in TypeScript the type of a variable cannot change after assignment.

JavaScript does not have an explicit dictionary class like Python's `dict`, so a common paradigm is to treat objects as dictionaries, and TypeScript needs to be able to type them as such without having to enumerate each of their possible entries, which is done with the `Record` type.

2.2.2 Properties of a class may depend on the constructor

While TypeScript allows for typed field declaration without explicitly making a constructor, it is syntactic sugar for a constructor that initializes the listed fields to their types' default values. The set of properties a class can have is the union of the properties returned from a constructor for all possible

inputs. Take, for example:

```
class Dog {
  constructor(hasbone) {
    this.goodboy = true
    if (hasbone) this.bone = new Bone()
  }
}
```

For a TypeScript program to allow for assignment in the constructor, all possible fields that could be assigned must be enumerated beforehand. Otherwise, TypeScript compiler will produce an error:

```
class Dog {
  goodboy: boolean
  bone?: Bone
  constructor(hasbone: boolean) {
    this.goodboy = true
    if (hasbone) this.bone = new Bone()
  }
}
```

2.2.3 TypeScript can't use generics with primitive operations

TypeScript is unable to perform "duck typing" with primitive operations (Figure 3). TypeScript ensures that any provided objects—which includes classes as they are a wrapper around objects—have necessary properties and methods to satisfy a given type signature, but is unable to do so with primitive operations like addition and multiplication.

The fact that this code snippet results in a compiler error is a limitation of

```
function add<T extends number|string>(a: T, b: T): T {
  return a + b // Operator '+' cannot be applied to types 'T' and 'T'
}
add(1, 2)
add("a", "b")
```

Figure 3: (See 2.2.3)

the TypeScript compiler. The type variable `T` ensures that when adding `a` and `b` both will share the same type—they will either both be a number or they will both be a string.

The best way to type this is to be somewhat unsafe and use both `any` and overloading, which satisfies the TypeScript compiler and results in correct code, but it means there is no type checking on either `a` or `b` within the function:

```
function add<T extends number|string>(a: T, b: T): T
function add(a: any, b: any): any {
  return a + b
}
add(1, 2)
add("a", "b")
add(1, "b")
// tsc: Argument of type '"b"' is not assignable to parameter of type '1'
```

2.2.4 Existence of multiple correct typings

In the case of type aliases, any type alias (or none at all) is a technically correct alias. Sometimes a collection of classes that share a parent class

can not be typed by the parent class alone. It can be difficult to determine whether we should type a function with its parent class or with the union of the subclasses.

```
interface Dog { color: string }
interface LoudDog extends Dog { bark: () => string }
interface MediumDog extends Dog { bark: () => string }
interface QuietDog extends Dog { }

function processdog(dog: Dog) {
  console.log(dog.color)
}

function processdog2(dog: LoudDog|MediumDog) {
  console.log(dog.color)
  console.log(dog.bark)
}
```

Types should be annotated with the least upper bound of the type lattice over the set of types that satisfy the TypeScript compiler. This results in the most concise types that are still correct. In this example, if we see objects all with `color`, but only some have `bark`, then it's a Dog, if all have both `color` and `bark`, then it can either be a LoudDog or a MediumDog but not any Dog or QuietDog.

In practice, since we are migrating JavaScript, these type definitions must come from packages. Both LoudDog and MediumDog look identical to any JavaScript function as both are simple objects. Static analysis can propogate the types of library functions that return interfaces, in order to help disam-

biguate types that satisfy multiple interfaces. Large language models could also be used to help differentiate based on semantic context.

2.3 Migration after types have been assigned

Most existing JavaScript type inference engines are not strictly made for JavaScript to TypeScript migration, but either just infer TypeScript types without checking the compiler [6, 10, 11, 23, 24], or make non-Typescript annotations [2, 15, 16]. It is possible to use these type inference engines.

To use the former for migration, non-ECMAScript imports must be made into module import syntax. `require` does not preserve type information and therefore must be avoided in TypeScript. There exist CommonJS to ECMAScript conversion tools, but do not generate perfect translations [26].

There also exist situations where untyped Node.js packages pass a class into a function, but do not export said class. Take the following code:

```
class Dog { ... }
export function makeDog() { return new Dog() }

// user code
import { makeDog } from "dog"
function petDog(dog) { dog.pet() }
// how do we type `dog` when `Dog` isn't exported?
let max = makeDog()
petDog(max)
```

In this case, we must create a TypeScript definition file for this package and,

in `tsconfig.json`, associate that definition file with the untyped package.

The TypeScript compiler is strict, so there are cases where it's not enough to simply annotate functions and variables. A meta-analysis of SOTA machine-learning-based inference type engines found that the most common of these errors seen when DeepTyper, LambdaNet, and InCoder were used for migration were properties not existing on types, assignment with mismatched types, incorrect function type labelling, calling non-functions, and conditional statements between mismatching types [26]. Some of this may be due to the nondeterministic nature of these probabilistic type inference engines, but some of this may strictly require a post-processing step from a TypeScript migration tool to `type import` the necessary classes.

3 Related work

3.1 Static analysis

Static type analysis typically uses data flow analysis to infer as many types as it can from the code [2, 11, 12]. Typically, this means transforming a given program's AST into some lower-level representation that encodes variable/property reads/writes, function calls/returns, branches, etc., upon which they can perform data flow analysis [11, 12]. Static analyses may only operate on a subset of JavaScript that is conducive to static analysis [2].

Static analysis works by continuously narrowing types. It starts by assuming all identifiers can take any value, then under this assumption applies any constraints it can, based on the content of the program. This first pass results in a new set of types for each identifier, which will be a subset of the old types and a superset of the true types. It repeats this process on the new assumed types until it finds the least fixed point.

3.2 Machine learning based analysis

ML-based analyses attempt to infer types probabilistically with machine learning.

One approach is a natural-language approach, using neural networks, inferring where types should be applied and what they should be applied to based on the surrounding text, similar to part-of-speech tagging [10]. Another sim-

ilar approach would be to simply feed it into an LLM built to fill in text, and have it fill in types where there did not exist any before [6]. An approach to this utilized by developers is an agentic loop, where the user provides both code and tests to an AI agent, and has it modify the code until it both passes all tests and the TypeScript compiler.

As an alternative to using the code itself, one approach is to use lexical context like comments and function/variable names to infer types [13] or even combine this lexical context with static analysis to collapse loose logical constraints [17].

Another method is to statically construct type dependency graphs. LambdaNet computes embeddings for each node, and then uses a graph neural network to predict each type in the graph [24]. JSNice [20] trains on these dependency graphs, including the names of the variables these types are associated with, and predicts not only the types but variable names, allowing them to undo minification.

These are not complete migration tools, and so a full JavaScript to TypeScript migration requires some extra instrumentation, and while they report good top-5 accuracy (the original type in the TypeScript was reported as one of the 5 most likely types), when actually applied to entire packages result in TypeScript that compiles only 20% of the time [26].

3.3 Dynamic analysis

Dynamic analysis runs the program and checks what types are present at runtime to determine the intended types of an unannotated function. To determine this, the code must be instrumented to log what types pass through each function and are assigned to each identifier. JavaScript can be instrumented with the debugger, by transforming the source code before execution to add instrumentation statements as TypeDevil does [15, 16, 19], or by modifying the JavaScript engine itself as in the case of the Mozilla JavaScript engine in Firefox, which uses runtime type analysis to produce more efficient bytecode when running JavaScript [8].

Instrumenting each individual call would add a significant amount of overhead and cause significant slowdowns. While typically dynamic analyses use all the information available to them, much of it is likely to be redundant, as functions may be repeatedly called with the same arguments. Only performing instrumentation on a sample of calls can result in significant speedups in execution time without significantly degrading the quality of reported types [18].

The efficacy of dynamic type inference is reliant on coverage. Any function that does not get executed cannot be annotated, so this method of type analysis is better for mature, well-tested codebases.

4 Implementation

In this paper we detail ScriptDoctor, a dynamic type migration tool for JavaScript. At a high level, ScriptDoctor observes functions' inputs and outputs at runtime, collecting a set of unique types seen, then emits TypeScript applying the observed types to the functions they were collected from.

4.1 Preprocessing

Dynamic type inference requires that the program be run in order to collect types. Every time a function is called, ScriptDoctor inspects the arguments of the function, as well as the function itself. ScriptDoctor does this via hooking into JavaScript's loading script using the node using the NodeJS `node:module` library, and transforming the source code as it gets imported, allowing for easy code instrumentation by specifying the environment variable `NODE_OPTIONS="--import /path/to/entry/script"`.

These code transformations must result in an equivalent program to the original, but many of the changes require adding to the namespace inside function bodies, since ScriptDoctor may need to mutate arguments to functions to better inspect them, and needs to keep track of a call identifier. To avoid namespace collisions, ScriptDoctor assigns all new identifiers unique names by generating a 36-character UUID.

4.1.1 Transformations

ScriptDoctor logs the types of inputs and outputs to functions by inserting profiling statements into the source code, as shown in Figure 4. ScriptDoctor may need to modify the contents of parameters before they are used in the rest of a function, so first replaces each with own unique identifier, and on the following line passes each parameter into a function that passes back the original values, along with a UUID. Two asynchronous functions called sequentially may not terminate in the same order that they were called, so the function `__logarg` generates a new UUID for each function call instance, which it then passes to all other logging functions, allowing ScriptDoctor to associate logs with calls asynchronously.

Destructuring of objects is no longer done within the parameter itself, but instead in the reassignment on the following line. Destructuring of arrays without capturing the rest of the array is done outside the function definition, so that the type of any elements not explicitly destructured are also captured.

ScriptDoctor passes functions' own objects as the second argument to `__logret`. In that way, internally, ScriptDoctor is able to associate the arguments to higher-order functions with functions already profiled by ScriptDoctor. ScriptDoctor cannot use function location to determine this, since after the code transformations, function location changes and so instead must associate function object with location. ScriptDoctor transforms anonymous functions to give them names by assigning them to a new variable inline.

Before transformation:

```
function add(a, b) {
        return a + b
}
```

After transformation:

```
function add(UUID1, UUID2) {
        let [UUID3, a, b] = global.__logarg("main.js:0", UUID1, UUID2)
        retrun global.__logret(UUID3, add, a + b)
}
```

Figure 4: Transformation of a simple addition function before execution. ScriptDoctor associates the types of the function parameters with both this unique call (whose ID is in `UUID3`) and the call site (`"main.js:0"`). Each `UUIDX` is notation representing a unique 36-character UUID (Section 4.1.1).

Since ScriptDoctor inserts lines into blocks in these transforms, ScriptDoctor transforms all expression function bodies into blocks (Figure 5).

4.2 Watched Objects

An object or function's true types can only be known for certain by the termination of the program, since at any point an object could have properties added or removed, and a function could be called with new arguments. Additional instrumentation is required to "watch" these types throughout the program's runtime, rather than just the lifetime of the function they were passed into.

Before transformation:

```
arr.filter(a => a > 5)
```

After transformation:

```
arr.filter(UUID1 = (UUID2) => {
        let [UUID3, a] = global.__logarg("main.js:12", UUID2);
        return global.__logret(UUID3, UUID1, a > 5);
})
```

Figure 5: Inline expression function transformed into a named block function. Note the assignment to a new UUID so ScriptDoctor is able to access the function object. Each `UUIDX` is notation representing a unique 36-character UUID (Section 4.1.1).

4.2.1 Functions

Functions from code that has not been instrumented (as in the case of any functions imported from `node_modules`) must have their types determined by means other than code transformation. ScriptDoctor instruments these functions by wrapping them as they are encountered. If `__logarg` sees a function that's not instrumented, it instead passes back a wrapped version that logs arguments, yields and returns. When the wrapper function is called, it logs all arguments before passing them on to the inner function. If the inner function returns a non-generator, that value is returned, and if it does return a generator, the generator is wrapped so that it reports all values yielded by it. Functions additionally are given a property with a unique ID that denotes it as already wrapped, so functions are not wrapped more than once.

If a function that has already been instrumented is passed into `__logarg`

there's no need to further instrument it. Any function that has been instrumented has a location associated with the call site (as passed in by `__logarg`) and one or more function objects associated with this location. If any function object matches one already associated with a location, it is not watched, letting ScriptDoctor accurately profile closures, where one function location may be associated with multiple different function objects.

4.2.2 Objects

An object's properties may be in flux during the program, but TypeScript requires all properties of the object to be enumerated at or before assignment. If a function has a property at one point in time and does not at another, it should be denoted as an optional property. If this is not done, the TypeScript compiler will return an error when assigning the new property. To observe property changes in objects, ScriptDoctor wraps each object with a proxy [1], notes any time a property is set or unset, and updates the recorded types accordingly, treating any new or unset properties as optional and any properties assigned with multiple types as their union.

ScriptDoctor marks these objects as wrapped as well so that objects are not double-wrapped by overwriting the `get` method as well. If `get` is called on the same unique identifier used for determining if functions are wrapped the object will return `true`. By marking it in this manner, the key is not actually added to the object so it is not listed when enumerating through keys.

```
for (let n=0; n<100; n++) {
        myfunction({n: 0})
}
```

Figure 6: Rather than treating this as a `Record<string, number>` each `n: 0` has a unique keyset and is therefore treated as a separate type when profiling, in case there is some supertype that can be inferred by the process shown in Section 4.4.3 (Sections 4.3, 4.4.3)

4.3 Data Collection

Since ScriptDoctor profiles every function call, were it to keep each call trace in memory then the memory cost of instrumentation would be prohibitively expensive. Memory usage would become a function of runtime. To avoid this cost, ScriptDoctor only preserves as little information about a given type as possible. Furthermore, for arguments to a function, only unique traces are stored rather than the entire call history. One limitation is that objects with varying keysets are each treated as different types (Figure 6), but this has not been a noticeable issue in testing, with memory footprints remaining relatively constant (Section 5.1.2).

4.3.1 Collecting Traces

When a function is first called, `__logarg` generates a unique "call ID" and corresponding function trace. Any yields or returns seen associated with this call ID have their types collected and added to the trace object. All yields and the eventual return associated with this call ID have their types collected and added to the trace object. When `__logret` is called, the function call

associated with this call ID has finished execution, and the trace is added to the set of unique observed call traces.

Each trace object contains a list of types for each of the function's parameters. Traces include optional parameters, which are marked as undefined. For generator functions, ScriptDoctor collects a list of all types yielded by the function and computes their union. Delegated yields are handled separately, since when returned by the original function they unwrap the delegatee. ScriptDoctor handles this by wrapping the delegatee, and each time `next()` is called, ScriptDoctor adds the value it returns to the set of types observed by yield (Figure 7).

4.3.2 Trace Sets

To ensure memory consumption is not unbounded, ScriptDoctor does not store identical traces. It can be difficult, however, to determine if two traces are identical. Two traces are identical if each argument has the same type in both traces. JavaScript's `typeof` operator is insufficient to determine type equality for objects, functions and classes. All objects and classes are of type `'object'` in the eyes of `typeof`, but classes can be typed by their name and objects with different keysets can be typed differently. Functions can be treated the same only if they have the same type signature, but said type signatures can only be determined after the program terminates and so each unique function object must be treated as a different type.

Before Transformation:

```
function* f() { yield* [1, 2, 3] }
```

After Transformation:

```
function* f() {
        let [UUID] = global.__logarg("file.js:0");
        // argument to a delegated yield must be a generator
        // which gets wrapped and its next() method is instrumented
        yield* gloabl.__logdelyield(UUID, [1, 2, 3]);
        return global.__logret(UUID, f, undefined);
}
f_gen = f();  // returns a generator
f_gen.next(); // next() call is delegated to [1, 2, 3] which is wrapped
              // number is added to the set of types yielded
```

Figure 7: Example data flow path for wrapped generators. Delegatees are wrapped in order to know what types are yielded without instrumenting the generator returned by the function, instead only instrumenting the function body (Section 4.3.1).

When ScriptDoctor observes a function return, it serializes the trace into a string and checks against a hash set if this trace has already been seen called by the function. If the trace is unique, it is added to the trace set. Storing only unique traces significantly decreases the memory consumption of the program.

ScriptDoctor can trivially serialize primitive types to strings, as `typeof` is sufficient, and serializes classes to the name of their constructor. Objects and functions are more difficult to serialize.

ScriptDoctor serializes functions to a unique ID either associated with the location or the function object. ScriptDoctor assigns an ID to all functions the first time they are called.

Objects are more complicated. To serialize an object's type, ScriptDoctor serializes all string and symbol keys of the object and all values each key takes. To serialize nested objects, ScriptDoctor performs a depth-first traversal over the object. Object references that have already been seen must not be visited again to prevent infinite recursion, but still must be uniquely serialized. When ScriptDoctor encounters a cycle in the object, it marks the child node as a reference to the parent rather than continuing traversal infinitely (Figure 8).

```
let outer = {}
outer.inner = outer
compute_typeinfo(outer).toUnique() // <ref *0>{inner:<Circular *0>}

outer = {}
outer.middle = {}
outer.middle.inner = outer.middle
compute_typeinfo(outer).toUnique() // {middle:<ref *0>{inner:<Circular *0>}}
```

Figure 8: Examples of cyclic objects and how they can be uniquely serialized to still be distinguishable. By keeping track of object references, if a depth-first traversal through the object sees a node its already visited, ScriptDoctor labels the inner object with a unique reference (Section 4.3.2).

4.3.3 TypeInfo Objects

ScriptDoctor does not keep the original function arguments, but rather a data object that represents the type itself, keeping as little information required to produce type annotations. These objects must be able to serialize themselves both to a unique string value and to a valid TypeScript type. ScriptDoctor serializes primitives using `typeof`. For non-primitives it is possible to differentiate between two of the same type due to metadata, such as what fields are in an object or the signature of a function.

4.4 Post-processing

During its post-processing phase, ScriptDoctor collects the various different traces, coalesces them into a single type for each identifier, and emits the updated TypeScript. If ScriptDoctor determines a type that does not exist natively in JavaScript but does in TypeScript (e.g. tuples) better suits an

identifier, it uses that type instead. ScriptDoctor additionally looks for parameters that are always called with the same types in order to infer generic types.

4.4.1 Computed Types

This paper defines computed as any TypeScript type that is not distinguishable by the `typeof` operator. Classes, tuples, unions and generics all constitute computed types.

Classes are distinguishable by their constructors. All non-class objects share the same constructor, specifically `Object.getPrototypeOf({}).constructor`. To determine if a given object is a class, ScriptDoctor first checks whether its constructor is not equal to the object constructor, and then checks if this constructor has a name. If it does not, and the constructor has been set to an anonymous function, ScriptDoctor treats it like it would an object.

If the class's constructor has a name, the name of the constructor is sufficient, but it is not guaranteed that the object is in module scope already, in which case ScriptDoctor must use `import type`, allowing for ScriptDoctor to use the class in types even if it has not been explicitly exported by the package.

In-package classes can use relative imports based on the file where the constructor is defined, but for classes from `node_packages`, ScriptDoctor checks the module exports from `package.json` to determine what import string should be used (Figure 9). In the case the file isn't found, ScriptDoctor

`package.json`:

```
{
  "exports": {
    ".": "./index.js",
    "./sync": "./sync.js"
  }
}
```

Before type inference:

```
import { makeDog, makeDogSync } from "simple_exports";
function f() { return makeDog() }
function g() { return makeDogSync() }
```

After type inference:

```
import type { Dog } from "simple_exports";
import type { DogSync } from "simple_exports/sync";
import { makeDog, makeDogSync } from "simple_exports";
function f(): Dog { return makeDog() }
function g(): DogSync { return makeDogSync() }
```

Figure 9: Automatic class imports from varying endpoints. Dog and DogSync come from different files, each requiring a different import string. ScriptDoctor automatically determines what input string should be used by parsing `package.json` and imports these classes (Section 4.4.1).

imports the absolute path as a fallback.

ScriptDoctor marks arrays as tuples if, whenever an array is passed into a parameter, it always has constant length below a user-defined threshold (default 5). ScriptDoctor treats each index of the tuple as its own identifier and each is typed independently.

4.4.2 Computing Generic Types

Generic type recognition is loosely based on the generic recognition algorithm from RightTyper [18]. The basic idea is to organize call traces into a matrix, where each row is a single call, and each column represents one identifier. If a subset of these identifiers are always called with values of matching types, we can safely assume that these values are generics, as false positives still result in a correct type signature. After transposing the matrix, row equality means two identifiers have the same type history and can therefore be parametrized.

Nested types like arrays, tuples and objects complicate this logic, as we want to treat each index of a tuple and each unique key of an object as a separate parameter, as they can be differentiated in the type signature. To separate these, this algorithm "flattens" these container types into the rows, effectively treating each as a different identifier. Flattening must be done recursively as well—generics should be identifiable even if we have an array of tuples. Flattening a column must result in a constant number of new columns, so each column still represents a type that can be differentiated.

Tuples have fixed size, so we simply unwrap n-length tuples into n separate columns, resulting in effectively n new identifiers.

If an array is non-homogeneous (i.e., it contains more than one type of element), ScriptDoctor does not attempt to parametrize its types as when flattening there is no guarantee on size. Homogeneous arrays are flattened to their inner type.

Objects require more attention. ScriptDoctor unwraps each unique key seen across all objects an identifier sees into its own column. If a given key does not show up in one call but does in another, it is set to `undefined`. Key order is lost when flattening these objects, so ScriptDoctor must preserve the order it unwrapped them in when it wraps them back up into objects.

4.4.3 Combining Types

Once ScriptDoctor has inferred computed types, it then needs to combine this list of possible types for a single identifier into a single type signature. The simplest method of doing this is to union all types together, and in most cases this is sufficient. For primitives, generics and even differing computed types, unioning is sufficient. In some cases, however, there may be better ways to combine types.

When combining objects, the set of all possible valid type annotations can be represented as a lattice whose partial ordering is determined by whether all keys of one object are present in another. To combine objects ScriptDoctor attempts to determine if there exists a least upper bound and, if one exists, ScriptDoctor is able to combine multiple different object signatures into one. Specifically, if there is some core subset of types which all objects observed within a single identifier share, we can be reasonably sure these objects share some similar structure, even though false positives are possible, and still result in correct type signatures. Computing this least upper bound detects JavaScript objects that act as classes with optional values.

ScriptDoctor also combines arrays of different types. Whether to do so or not is a design question—`number[]|(number|string)[]` is more verbose than `(number|string)[]` but more specific; ScriptDoctor opts for the former as it provides more information about possible array structure.

4.4.4 TypeScript Generation

Once types have been combined, ScriptDoctor transforms the source abstract syntax tree using a package that supports both JavaScript and TypeScript, parsing the original JavaScript and emitting TypeScript with annotated functions. To produce these annotations, the computed type info objects are recursively serialized into type strings.

During ScriptDoctor's in-order traversal of the abstract syntax tree, to ensure there are no name collisions in generics, it keeps track of how many generics are present in the namespace (Figure 10).

If a library function returns a class, it may be necessary to import the type as a class too. ScriptDoctor imports this class at the top of the TypeScript file (Figure 9).

4.5 Usage

For ScriptDoctor to extract types, it must observe high-coverage execution. There are two primary methods of doing this. The first is to profile the program as normal for an extended period of time. If the program is well-

```
function b<T0 extends number|string>(
                a: <T1 extends number|string>(p0: T1) => T1,
                s: T0
): T0 {
        return a(s);
}
function a<T0 extends number|string>(s: T0): T0 { return s; }
```

Figure 10: When there is already a generic `T0` in-scope, ScriptDoctor does not overwrite it. Once `T0` leaves scope it is once again available as a type parameter name, resulting in cleaner generic names (Section 4.4.4).

utilized in this time such that all execution paths leading to unique type inputs and outputs for functions are covered, the types ScriptDoctor emits will correctly describe the functions input and output types.

Unit tests are a proxy for profiling a program over a long period of time. Tests have the same goal of effectively executing the program with high coverage and testing various execution paths and function input types in order to execute as many edge cases as possible, so profiling a package over its unit tests is often sufficient to perform a good type migration.

4.6 Evaluation

This paper evaluates ScriptDoctor both over small code snippets or "litmus tests" that assert it can correctly type various minimal examples, and whole package tests where a package's tests are run with ScriptDoctor's instrumentation. This allows evaluation of types against a ground truth, as well as determining memory and speed overheads.

5 Results

This paper evaluates ScriptDoctor over a small selection of packages from the TypeWeaver [26] dataset of typed packages with no dependencies, as well as a selection of code snippets of varying complexity. Performance is compared against DeepTyper [10], InCoder [6], SantaCoder [3] and LambdaNet [24], and litmus tests are compared against Claude Opus 4.7, DeepTyper and InCoder.

We evaluate ScriptDoctor against SOTA along three dimensions:

- **Research Question 1:** What is the overhead of ScriptDoctor?
- **Research Question 2:** How effective is ScriptDoctor at typing both common and uncommon JavaScript patterns?
- **Research Question 3:** How effective is ScriptDoctor at typing complete packages?

5.1 Research Question 1: Overhead

Performance for all type inference engines was measured on 2x 32-core Intel Xeon Gold 6430 with a maximum clock speed of 3.400GHz and both an NVIDIA RTX 6000 Ada Generation and an NVIDIA Quadro RTX 6000.

5.1.1 Speed

Program duration is determined using `time npm run test` both with and without instrumentation and real time is reported. Duration for all other type inference engines is determined by prefixing docker entry points with `time -v`.

Package	Base	ScriptDoctor	DeepTyper	InCoder	LambdaNet
ini	1.867s	4.893	10.7778s	16.20s	DNF
abab	0.600s	2.409s	12.693s	19.07s	68.347s
dotenv	2.839s	10.937s	16.67s	10.617s	66.541s
node-int64	0.306s	1.716s	10.555s	13.47s	67.772s

Line of best fit for ScriptDoctor with R^2 of 0.9211 (y is instrumented time in seconds, x is uninstrumented time in seconds):

$$y = 3.432 * x + 0.1737$$

This implies that both preprocessing and post-processing took approximately 0.1737 seconds, and the instrumentation overhead is approximately 243.2%.

ScriptDoctor is faster than all other type inference engines. While ScriptDoctor's runtime scales with duration of tests rather than program size, machine-learning-based type inference engines require a significant amount of time both to load their models' weights and to perform inference, resulting in ScriptDoctor being faster.

5.1.2 Memory consumption

Memory usage for ScriptDoctor is determined using `valgrind --tool=massif npm run test` both with and without instrumentation. Maximum of uninstrumented unit tests and both greatest local maximum and second-greatest local maximum of instrumented unit tests are reported. Memory usage for all other type inference engines is determined by prefixing docker entry points with `time -v`.

Package	Tests	Max ScriptDoctor	2nd ScriptDoctor	DeepTyper	InCoder	LambdaNet
ini	3.6 MiB	36.1 MiB	23.7 MiB	1 GB	14 GB	DNF
abab	3.5 MiB	36.1 MiB	22.8 MiB	713 MiB	14 GB	2.7 GB
dotenv	3.6 MiB	36.3 MiB	23.3 MiB	712 MiB	13.9 GB	2.6 GB
node-int64	3.5 MiB	36.1 MiB	27.7 MiB	961 MiB	13.9 GB	2.9 GB

In all four cases, ScriptDoctor's memory graphs were largely identical (see figures 11 and 12 for examples). The uninstrumented code increases until about 3.5 or 3.6 MiB at which point some memory is freed. Usage is likely dominated by the node runtime itself rather than the test runner or the unit tests, as all four packages have approximately the same memory footprint. The instrumented code peaks at 36.1–36.3 MiB at the start and then peaks again towards the end at 22.8–27.7 MiB. The first of these peaks corresponds to initial instrumentation and code transforms, and the second to post-processing and writing out the resulting TypeScript program.

ScriptDoctor has a drastically lower overhead compared to all other measured type inference engines since it does not have to load a model in-memory, but

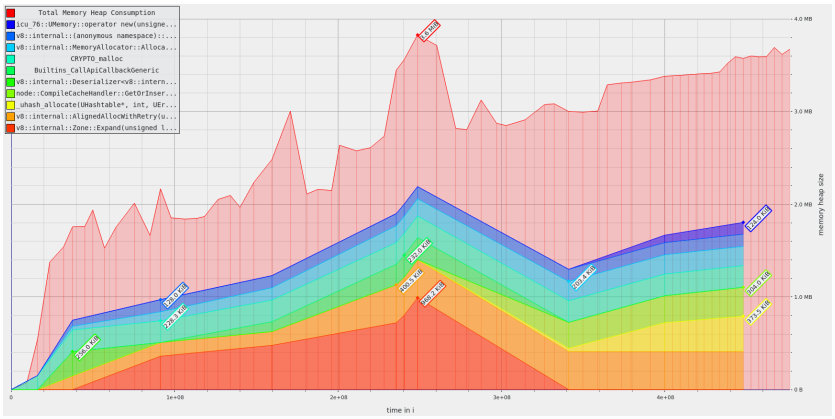


Figure 11: Memory usage for `npm run test` on package `dotenv` with no instrumentation, as visualized with massif-visualizer. The red, transparent region represents total memory heap consumption for the program. Memory increases until reaching its maximum of 3.6 MiB (See 5.1.2).

only takes an additional ≈ 33 MiB.

5.1.3 Research Question 1: Summary

ScriptDoctor both uses drastically less memory and is significantly faster than other state-of-the-art type inference tools. Its peak memory footprint of ~ 36 MiB is 19.8–387.8 times smaller than the ML-based engines, which range from hundreds of megabytes (DeepTyper) to 14 GB (InCoder). ScriptDoctor is up to 39.5 times faster, with its advantage stemming from avoiding the fixed cost of loading model weights and inference overhead that the ML-based engines incur.

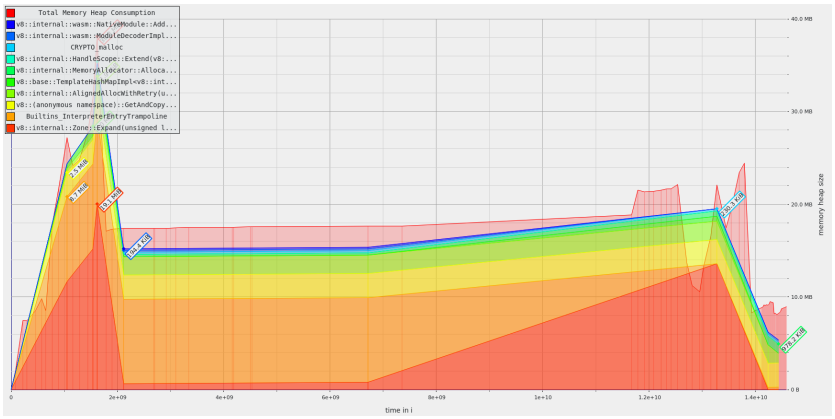


Figure 12: Memory usage for `npm run test` on package `dotenv` with instrumentation, as visualized with massif-visualizer. The red, transparent region represents total memory heap consumption for the program.. Memory peaks at 36.1 MiB at the start—likely during source transformations and importing ScriptDoctor code—and then again towards the end—likely when coalescing types in post-processing (See 5.1.2).

5.2 Research Question 2: Litmus Tests

This paper presents selection of cases showing a subset of behavior that is nontrivial to type. We evaluate against DeepTyper [10], InCoder [6] and Claude Opus 4.7. Claude is closer to ground truth and is quite effective at assigning types in litmus cases like these, though there are principled decisions in this work that differ from what Claude finds. The prompt used is: `"Transform this JavaScript code into TypeScript. Output only the code. {code}"`. While this is a one-shot prompt, large language models excel at small examples with minimal context; we find this is sufficient to obtain good types.

5.2.1 Object assignment

```
function f(a) {
  a.b = 1
  return a
}
f({a: 1})
```

This function takes arbitrary objects and ensures that the resultant type has property `b` assigned to a number. The function is only ever passed objects that have property `a` assigned to a number.

ScriptDoctor:

```
function f(a: { a: number, b?: number }): { a: number, b?: number } {
  a.b = 1
  return a
}
```

ScriptDoctor finds ideal types. It is necessary to mark `b` as optional when assigning it, so that both we can pass in `{ a: 1 }` without a type error, and so that we are able to assign to `b` within the function. Arguably a more ergonomic typing would return `{ a: number, b?: number }` but per TypeScript convention, object types are assumed not to change throughout the runtime of the program, so should we have not done this, the following would result in a type error, as mentioned in Section 2.2.1:

```
let a = { a: 1 }
f(a)
```

```
console.log(a.b) // type error, b does not exist on type { a: number }
```

Claude Opus 4.7:

```
interface Obj {
  a?: number;
  b?: number;
}

function f(a: Obj): Obj {
  a.b = 1;
  return a;
}
```

Claude finds more permissive types. Claude effectively finds the same type, but needlessly marks parameter `a` as optional and creates an extraneous interface that could have been written in-line. While this is not strictly negative, Claude unnecessarily increases the size of the program for dubious gains in readability.

DeepTyper:

```
function f : string(a : any) {
  a : any.b : number = 1
  return a : string
}
```

DeepTyper's types are not internally consistent. It's unclear why DeepTyper sees argument `a` as any in the parameter and as a string when returned, but DeepTyper is unable to accurately type code snippets of this

form as it does not support complex object types.

InCoder:

```
function f(a: A) {
  a.b = 1
  return a
}
```

InCoder hallucinates. InCoder fails to determine a return type for the function and hallucinates a class `A`.

5.2.2 Complex nested generics

```
function c(s) {
  function a(s) { return s; }
  function b(f, s) {
    return f(s);
  }
  return b(a, s)
}
c(5);
c("a");
```

Here we have a variety of no-op functions: `a`, which simply passes back a function, `b` which calls its first parameter on its second, and `c` which calls `b` on `a` and its input, resulting in another no-op. All arguments should be parametrized, and the generic can be specified to be either a number or a string while preserving type correctness.

ScriptDoctor:

```
function c<T0 extends number|string>(s: T0): T0 {
  function a<T1 extends number|string>(s: T1): T1 { return s; }
  function b<T1 extends number|string>(
    f: <T2 extends number|string>(p0: T2) => T2, s: T1
  ): T1 {
    return f(s);
  }
  return b(a, s)
}
```

ScriptDoctor finds valid types, but does not associate generics. ScriptDoctor marks each generic separately since it treats functions as entirely independent of one another. Note that `a`, `b` and `c` all have the same generics, but ScriptDoctor is unable to link them, and doing so would be difficult using dynamic analysis alone. Note the function parameter to `b` inherits the type of `a`.

Claude Opus 4.7:

```
function c<S>(s: S): S {
  function a(s: S): S { return s; }
  function b(f: (s: S) => S, s: S): S {
    return f(s);
  }
  return b(a, s);
}
```

Claude associates generics, but does not specify their type. Claude finds the same generics but links them all together. It decides not to explicitly mark the types as either a number or string which given the perceived

intention of the code (to be a no-op) is a valid choice, since the code itself places no constraints on the types of the parameters.

DeepTyper

```
function c : void(s : string) {
  function a : string(s : string) { return s : string; }
  function b : any(f : string, s : string) {
    return f : string(s : string);
  }
  return b : void(a : string, s : string)
}
```

DeepTyper struggles with higher-order functions and unions. While string is a valid subset of types in most cases, `f` is certainly not a string and `c` certainly is not a void function, resulting in multiple errors found by the TypeScript compiler.

InCoder

```
function c(s: string) {
  function a(s: string) { return s; }
  function b(f: (a: number) => number,  s: number) {
    return f(s);
  }
  return b(a, s)
}
```

InCoder finds neither generics nor unions. InCoder performs significantly better than DeepTyper, finding that f is a function, but still has a hard time with unions, opting neither to pick one type nor use a union, but

instead using both types in different places. InCoder's transformed code results in multiple errors found by the TypeScript compiler, when calling `b` with `a`, `f` with `s`, and when calling `c` with a number.

5.2.3 Simple addition

```
function add(a, b) {
  return a + b
}
add(1, 1);
add("a", "b");
```

Here we have a simple multi-type addition function. This function can be passed any JavaScript objects that support addition, but is particularly difficult to type for two reasons; First it requires the type engine to able to handle generics (or something of the sort). Second, generics alone are insufficient due to limitations of TypeScript's static analysis.

ScriptDoctor:

```
function add<T0 extends number|string>(a: T0, b: T0): T0 {
  return a + b;
}
```

ScriptDoctor finds a valid generic, but results in a type error. As mentioned, this is insufficient as TypeScript treats `T0` as `number|string`, rather than both `a` and `b` being either both numbers or both strings, and disallows adding `a` and `b` together, resulting in a type error. TypeScript's

issues with generics over primitive operators are mentioned in Section 2.2.3.

Claude Opus 4.7:

```
function add(a: number, b: number): number;
function add(a: string, b: string): string;
function add(a: any, b: any): any {
  return a + b;
}
```

Claude has no type errors, but has no type checking for the function body. What Claude comes up with has its own tradeoffs, as while it does not result in a TypeScript error, it forgoes any type checking within the function, as while the overrides ensure that `a` and `b` are called with the same type, the TypeScript compiler effectively treats them like `any`.

DeepTyper:

```
function add : number(a : string, b : string) {
  return a : number + b : string
}
```

DeepTyper mixes strings and numbers. DeepTyper turns two string parameters into a string and a number resulting in the function returning a number. This would throw type errors for both invocations of `add`, and when computing `a + b` since TypeScript disallows addition of numbers and strings.

InCoder:

```
function add(a: number,  b: number) {
  return a + b
}
```

InCoder discards string concatenation. InCoder decides the parameters to be numbers instead and neglects to include a return value, though this is sufficient for the TypeScript compiler to infer the return type as number, but fails to account for string addition and will throw a TypeScript error for the second invocation of `add`.

5.2.4 Generators

```
function* f() { yield 3; return "hi" }
function* g() { yield* [1, 2, 3] }
```

This code snippet is a simple test for somewhat underutilized JavaScript syntax. The snippet tests generators, simple yields, returning from generators, and delegated yields.

ScriptDoctor:

```
function* f(): Generator<number, string> { yield 3; return "hi" }
function* f(): Generator<number> { yield* [1, 2, 3] }
```

ScriptDoctor finds correct types. ScriptDoctor correctly types both forms of generators, correctly unwraps the type of the delegated yield,

Claude Opus 4.7:

```
function* f(): Generator<number, string, unknown> { yield 3; return "hi"; }
function* g(): Generator<number, void, unknown> { yield* [1, 2, 3]; }
```

Claude finds correct types. Claude finds the same transformation, but opts to explicitly return empty returns and unknown thrown errors. It was a design choice not to include these in ScriptDoctor for more concise types, but whether to explicitly set default types is up to developer preference.

DeepTyper:

```
function* f() { yield 3; return "hi" }
function* g() { yield* [1, 2, 3] }
```

DeepTyper is unable to annotate this code snippet.

InCoder:

```
function* f() { yield 3; return "hi" }
function* g() { yield* [1, 2, 3] }
```

InCoder is unable to annotate this code snippet.

5.2.5 Research Question 2: Summary

This work consistently outperforms related work in its type inference capability. Furthermore, this work is on par with a state-of-the-art LLM coding agent in these extremely simple cases with minimal context required, which LLMs already excel at, but due to the nature of dynamic type inference

also generalizes to larger programs (Section 5.3) and ensures logical equivalence between the transformed program and the original, which LLMs are not guaranteed to do (Figure 1).

5.3 Research Question 3: Full package evaluation

We evaluate ScriptDoctor, DeepTyper, InCoder, LambdaNet and SantaCoder over entire packages and keep track of the number of errors found by the TypeScript compiler, the number of times `any` is used in function annotations, and the total number of functions profiled.

5.3.1 abab

As the functions in `atob` and `btoa` are called with a variety of types (`number`, `string`, `null`, `undefined`, arbitrary objects with a `toString` method) only to narrow them all to a string using `argument=‘$argument‘`, ScriptDoctor has more errors from the TypeScript compiler, as TypeScript does not update the types of variables when they have been reassigned. To correctly type this, variables would have to be re-declared with new type signatures effectively rewriting the code in static single assignment (SSA) form. DeepTyper, LambdaNet and InCoder mark the input parameter as a string and thus TypeScript sees no type narrowing. LambdaNet sees the fewest because it annotated an array, initally marked as undefined and later assigned to numbers, as `any[]` (Table 1).

Package	Engine	TSC errors	`any` count	Functions profiled
abab	ScriptDoctor	40	0	4
	DeepTyper	21	0	4
	InCoder	13	1	4
	LambdaNet	9	0	4
	SantaCoder	12	0	4
dotenv	ScriptDoctor	18	0	6
	DeepTyper	9	3	7
	InCoder	10	7	7
	LambdaNet	29	2	7
	SantaCoder	9	2	7
node-int64	ScriptDoctor	19	0	10
	DeepTyper	15	10	11
	InCoder	22	0	11
	LambdaNet	41	0	11
	SantaCoder	22	2	11
ini	ScriptDoctor	34	0	6
	DeepTyper	13	6	7
	InCoder	16	0	0
	SantaCoder	16	0	0

Table 1: Full package evaluation results combining all four packages. LambdaNet is omitted for `ini` as it cannot parse its `package.json`.

5.3.2 dotenv

ScriptDoctor misses annotating one function since the test harness `tap` sandboxes groups of tests, resulting in individual runs only running a subset of the tests as not all functions are covered in each test run. Dotenv checks existence of properties via the common JavaScript pattern of `obj && obj.property`; DeepTyper and InCoder type this as `any` and SantaCoder hallucinates a nonexistent type, so the property existence check does not result in additional errors, whereas both ScriptDoctor and LamdaNet type them as objects, inducing extra type errors (Figure 13, Table 1).

5.3.3 node-int64

One function was not called in testing (an inspect method) and so ScriptDoctor was unable to profile it, though as it directly returns a string this did not result in a type error, and all other type inference engines opted not to explicitly type this function either. InCoder mistakenly outputs `igInteger` instead of `BigInteger`, but had it written `BigInteger` it would have only added 4 more type errors. `Int64`s have multiple ways they can be constructed, and ScriptDoctor enumerates 4/5 of them in the constructor as only 4/5 of them are tested. All others only allow for a single type in the constructor (Table 1).

5.3.4 ini

Both InCoder and SantaCoder did not apply any types to the entire package. This may be because all functions were anonymous, effectively typing

ScriptDoctor:

```
function config (options: { path: string }
    |{ encoding: string }
    |{ debug: string }
    |{ override: boolean }
    |{ debug: boolean }
    |{ debug: boolean, override: boolean }
    |undefined
): { parsed: { test: string } }|{ error: Error } {
  ...
  const debug = Boolean(options && options.debug)
  // property debug does not exist on type ...
  const override = Boolean(options && options.override)
  // property override does not exist on type ...
  ...
}
```

InCoder:

```
function config (options: any) {
  ...
  const debug = Boolean(options && options.debug) // no issues
  const override = Boolean(options && options.override) // no issues
  ...
}
```

Figure 13: InCoder does not contend with property accesses in the function body as it simply annotates. ScriptDoctor finds no unique least upper bound for the types its seen and thus treats them as separate (See 5.3.2).

all parameters with `any`. DeepTyper's heavy use of `any` obfuscated type errors that come from property accesses similarly to `dotenv` (Section 5.3.2). LambdaNet is omitted as it is unable to parse `ini`'s `package.json` (Table 1)

5.3.5 Research Question 3: Summary

ScriptDoctor frequently results in more errors from the TypeScript compiler than alternatives. ScriptDoctor does not ever use `any`, which significantly increases type errors, but always results in more meaningful type annotations,as `any` obscures possible type errors. ScriptDoctor frequently produces more complex types than other type inference engines—ScriptDoctor could drastically reduce the number of type errors by typing all objects as `object` or `any` and only choosing the single most frequent type seen instead of unioning all seen types, but doing so would be less effective for type migration and obfuscate potential runtime errors.

Frequently JavaScript developers will allow for multiple input types, but use type coercion to treat them as one type (see Section 5.3.1). Any type inference engine that predicts this one type will result in significantly fewer type errors, but would be an inaccurate representation of the function's capabilities. While ScriptDoctor would frequently output unions of types, all other type inference engines would only predict one type, resulting in fewer errors returned by the TypeScript compiler, but are inaccurate type predictions.

6 Conclusion

This paper presents ScriptDoctor, a novel approach to type inference and type migration for JavaScript. Where prior work has largely focused on static analysis and machine-learning-based approaches, we exhibit a wholly dynamic type inference engine. ScriptDoctor is significantly faster than state-of-the-art machine-learning based type inference engines, has better support for modern JavaScript features and frequently predicts better types.

6.1 Future Work

There are some improvements that could be made to the type combination algorithm and the generic computation algorithm. Combining objects could use an algorithm similar to QuAC's [25]. Heterogeneous arrays may be able to have some of their elements parametrized. If importing code from a definitely typed package, ScriptDoctor could scan through the interfaces it defines to check if any objects seen in the file match their structure.

Due to time constraints, ScriptDoctor was only evaluated over a small subset of packages of the TypeWeaver dataset. In future work, ScriptDoctor can be integrated with TypeWeaver in order to more comprehensively evaluate the types it generates.

Overhead can be reduced significantly from 243.2% by using a sampling strategy similar to RightTyper. Currently ScriptDoctor traces every single function call, but as shown in RightTyper it is possible to accurately

profile types by only performing instrumentation for a fraction of the calls. ScriptDoctor is likely unable to run with as little overhead as RightTyper, however, since python allows for significantly more efficient tracing with their `sys.monitoring` api while ScriptDoctor has to use code transforms.

Variable assignments can be profiled, both to reduce TypeScript compiler errors in the function body and to detect when a variable's type has changed and create a new variable for the updated type, which would eliminate the kind of error seen in Section 5.3.2.

6.2 Artifacts

ScriptDoctor is publicly avaiable at https://github.com/andOrlando/thesis

7 References

[1] 2026. Proxy - JavaScript — MDN. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Proxy. Accessed: 2026-05-13.

[2] Christopher Anderson, Paola Giannini, and Sophia Drossopoulou. 2005. Towards Type Inference for JavaScript. In *ECOOP 2005 - Object-Oriented Programming*, Andrew P. Black (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 428–452.

[3] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel Lamy Poirier, Hailey Schoelkopf, Sergey Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Lappert, Francesco De Toni, Bernardo García del Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luis Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. 2023. SantaCoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988* (2023). doi:10.48550/arXiv.2301.03988

[4] Gavin Bierman, Martín Abadi, and Mads Torgersen. 2014. Understanding TypeScript. In *ECOOP 2014 – Object-Oriented Programming*,

Richard Jones (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 257–281.

[5] Avik Chaudhuri, Panagiotis Vekris, Sam Goldman, Marshall Roch, and Gabriel Levi. 2017. Fast and precise type checking for JavaScript. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 48 (Oct. 2017), 30 pages. doi:10.1145/3133872

[6] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. arXiv:2204.05999 [cs.SE] https://arxiv.org/abs/2204.05999

[7] Zheng Gao, Christian Bird, and Earl T. Barr. 2017. To Type or Not to Type: Quantifying Detectable Bugs in JavaScript. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 758–769. doi:10.1109/ICSE.2017.75

[8] Brian Hackett and Shu-yu Guo. 2012. Fast and precise hybrid type inference for JavaScript. *SIGPLAN Not.* 47, 6 (June 2012), 239–250. doi:10.1145/2345156.2254094

[9] Stefan Hanenberg, Sebastian Kleinschmager, Romain Robbes, Éric Tanter, and Andreas Stefik. 2013. An empirical study on the impact of static typing on software maintainability. *Empirical Software Engineering* 19 (10 2013). doi:10.1007/s10664-013-9289-1

[10] Vincent J. Hellendoorn, Christian Bird, Earl T. Barr, and Miltiadis Allamanis. 2018. Deep learning type inference. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Lake Buena Vista, FL, USA) *(ESEC/FSE 2018)*. Association for Computing Machinery, New York, NY, USA, 152–162. doi:10.1145/3236024.3236051

[11] Simon Holm Jensen, Anders Møller, and Peter Thiemann. 2009. Type Analysis for JavaScript. In *Static Analysis*, Jens Palsberg and Zhendong Su (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 238–255.

[12] Tobias Kahlert. 2018. *Data Flow Based Type Inference for JavaScript*. Master's thesis. Karlsruhe Institute of Technology. https://pp.ipd.kit.edu/uploads/publikationen/kahlert18masterarbeit.pdf

[13] Rabee Sohail Malik, Jibesh Patra, and Michael Pradel. 2019. NL2Type: Inferring JavaScript Function Types from Natural Language Information. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 304–315. doi:10.1109/ICSE.2019.00045

[14] Michael Mathews and JSDoc Contributors. 2011. JSDoc: An API documentation generator for JavaScript. https://jsdoc.app/. Accessed: 2026-05-18.

[15] Nico Naus. 2015. Dynamic type inference for JavaScript. https://studenttheses.uu.nl/handle/20.500.12932/22912

[16] Morten Passow Odgaard. 2014. Javascript type inference using dynamic analysis. *Master's thesis, Aarhus University* (2014). https://scg.unibe.ch/assets/archive/external/JavaScriptTypeInferenceUsingDynamicAnalysisMasterThesis.pdf

[17] Irene Vlassi Pandi, Earl T. Barr, Andrew D. Gordon, and Charles Sutton. 2021. OptTyper: Probabilistic Type Inference by Optimising Logical and Natural Constraints. arXiv:2004.00348 [cs.PL] https://arxiv.org/abs/2004.00348

[18] Juan Altmayer Pizzorno and Emery D. Berger. 2025. RightTyper: Effective and Efficient Type Annotation for Python. arXiv:2507.16051 [cs.PL] https://arxiv.org/abs/2507.16051

[19] Michael Pradel, Parker Schuh, and Koushik Sen. 2015. TypeDevil: Dynamic Type Inconsistency Analysis for JavaScript. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. 314–324. doi:10.1109/ICSE.2015.51

[20] Veselin Raychev, Martin Vechev, and Andreas Krause. 2015. Predicting Program Properties from "Big Code". *SIGPLAN Not.* 50, 1 (Jan. 2015), 111–124. doi:10.1145/2775051.2677009

[21] Sergii Rudenko. 2020. ts-migrate: A Tool for Migrating to TypeScript at Scale. https://medium.com/airbnb-engineering/ts-migrate-a-tool-for-migrating-to-typescript-at-scale-cd23bfeb5cc.

[22] Fabian Schiebel, Florian Sattler, Philipp Dominik Schubert, Sven Apel, and Eric Bodden. 2024. Scaling Interprocedural Static Data-Flow Analysis to Large C/C++ Applications: An Experience Report. In *38th European Conference on Object-Oriented Programming (ECOOP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 36:1–36:28. doi:10.4230/LIPIcs.ECOOP.2024.36

[23] Yuli Wang, Xiaoqing Gong, Hao Chen, Jing Li, Baoying Liu, and Shuai Cao. 2020. JSTIFuzz: Type-Inference-based JavaScript Engine Fuzzing. In *2020 International Conference on Networking and Network Applications (NaNA)*. 381–387. doi:10.1109/NaNA51271.2020.00071

[24] Jiayi Wei, Maruth Goyal, Greg Durrett, and Isil Dillig. 2020. LambdaNet: Probabilistic Type Inference using Graph Neural Networks. arXiv:2005.02161 [cs.PL] https://arxiv.org/abs/2005.02161

[25] Jifeng Wu and Caroline Lemieux. 2024. QuAC: Quick Attribute-Centric Type Inference for Python. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 343 (Oct. 2024), 30 pages. doi:10.1145/3689783

[26] Ming-Ho Yee and Arjun Guha. 2023. Do Machine Learning Models Produce TypeScript Types That Type Check?. In *37th European Conference on Object-Oriented Programming (ECOOP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 263)*, Karim Ali and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 37:1–37:28. doi:10.4230/LIPIcs.ECOOP.2023.37